

自作3Dゲームエンジンにおける 「動的物理とメモリ最適化」の実装

個人製作ポートフォリオ技術発表資料

LE3B_09_コウダ_アユ

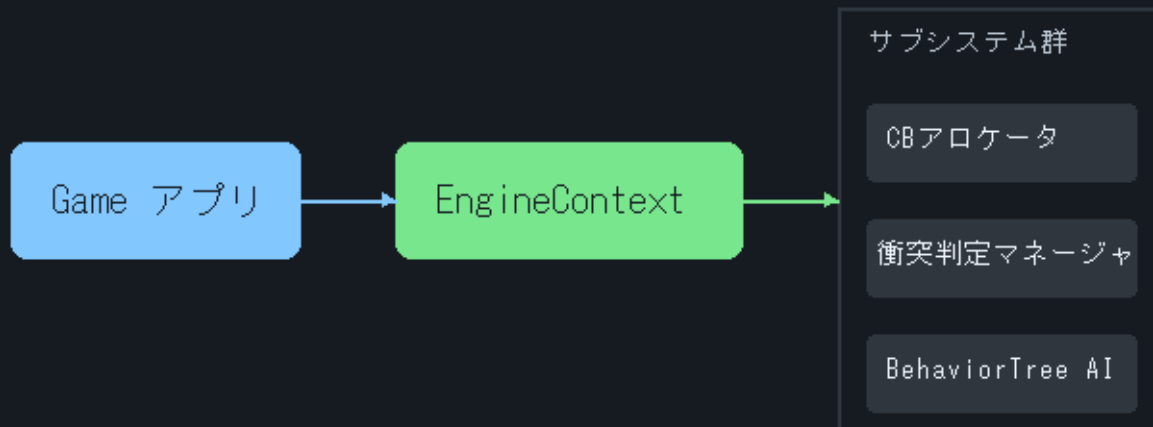
1. プロジェクトの概要

- 自作エンジン「Baziru3 Engine」の構築

- C++ & DirectX 12 でスクラッチ開発。

- テーマ：動的物理とメモリの最適化

- コライダー、AI、メモリを低レイヤで疎結合化。
- コンソール開発で重要となる「動的確保の抑制」と「計算量削減」を実証。



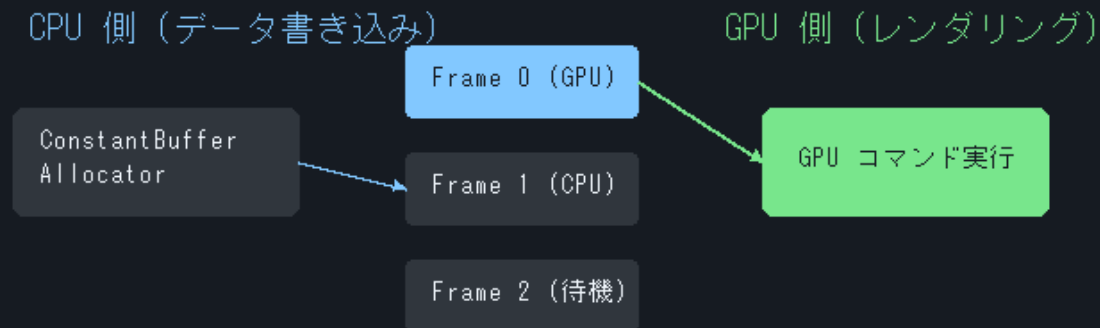
2. 解決すべき技術的課題

多数のオブジェクトやAIを同時動作させるための負荷対策

- 課題1：動的メモリ確保（new/delete）のオーバーヘッド
 - 毎フレームのリソース生成によるメモリ断片化と処理落ちの防止。
- 課題2：衝突判定の計算爆発 ($O(N^2)$)
 - キャラクター増加に伴う、総当たり当たり判定の処理重化。
- 課題3：開発調整イテレーションの遅さ
 - パラメータやAIロジック変更のたびに再ビルドが発生する。

3. 定数バッファアロケータの最適化

- 課題：毎フレームの定数バッファリソース生成によるアロケーション負荷
- 対策： **ConstantBufferAllocator** によるメモリ一括切り出し
 - 3フレーム分の定数バッファ領域を、起動時に一括で確保。
 - 256バイト境界アラインを自動調整し、ポインタ前進のみで高速切出。
 - GPUが読み込み中のバッファへのCPU上書きを防ぐフェンス同期。

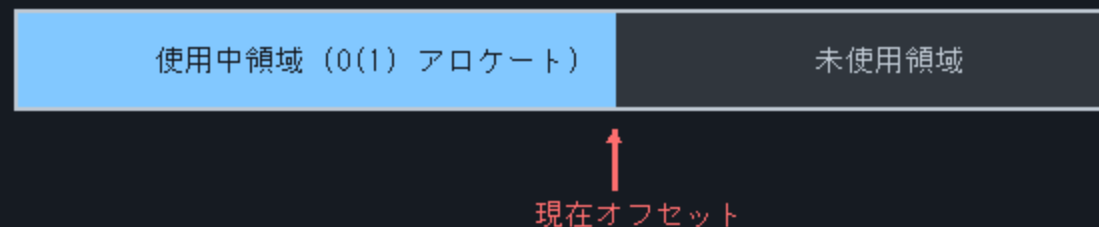


【GPU フェンス値と同期し、書き込みの衝突を防止】

4. スタックアロケータによる一時メモリ管理

- 課題：更新ループ中のテンポラリデータ確保によるメモリ断片化
- 対策： `StackAllocator` の実装
 - 起動時に一括で大容量メモリプールを確保。
 - 確保 ($O(1)$)：内部オフセットポインタをサイズ分進めるだけの高速処理。
 - 解放 ($O(1)$)：フレーム終了時にポインタを「0」に戻すだけの一括リセット。

起動時に一括でプール確保 → フレーム終了時に一括Reset()



5. 空間ハッシュによる衝突判定の高速化

- 課題：オブジェクト増加に伴う総当たり衝突判定の処理重化
- 対策： **SpatialHashCell** による空間分割 of 衝突判定
 - 3D空間を **10.0f** ごとの3Dグリッドセルに分割。
 - 座標からハッシュキーを算出し、近隣セル内のオブジェクト同士のみを判定。

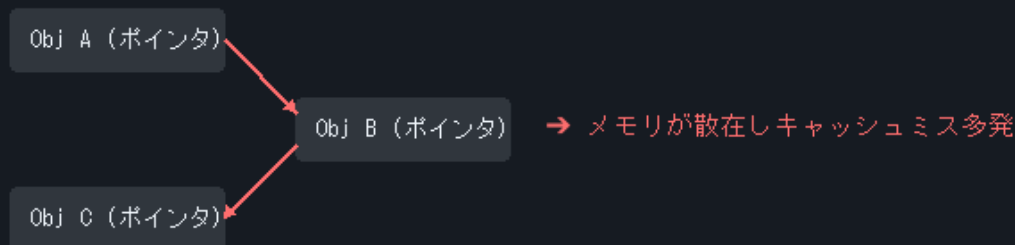


→ ワールド座標 / セルサイズ
→ セル所属判定
→ 同一・近隣セル内のみ判定
→ 計算量を $O(N^2)$ から $O(N)$ へ

6. データ指向設計（DOD）によるキャッシュ効率化

- 課題：オブジェクト指向によるメモリ散らばりとキャッシュミス
- 対策： **CollisionData** 配列のメモリ連続配置
 - 判定に必要なパラメータ（座標、半径等）のみを連続配列に抽出。
 - メモリ上で隣接した配列に対して、単純ループで一括判定処理。

従来の設計（OOP：オブジェクト指向）



最適化設計（DOD：データ指向設計）

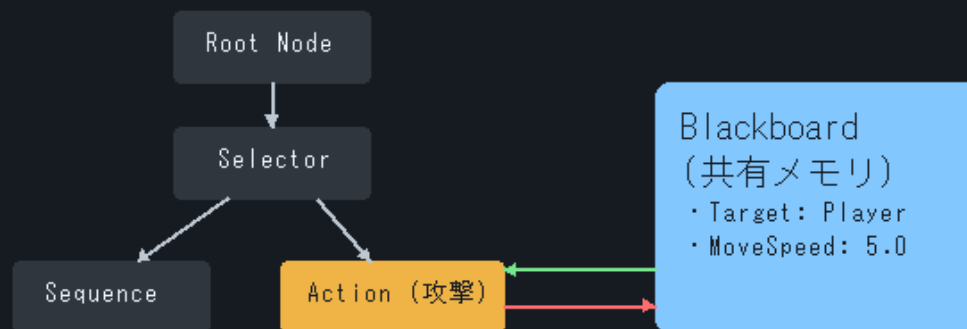
Data A	Data B	Data C	連続メモリ
--------	--------	--------	-------

7. 精密な衝突判定と多様なコライダー形状

- ゲーム性に応じた高度な当たり判定の実装
- 多様なコライダータイプ:
 - Sphere / Box (OBB・AABB) / Capsule: 標準形状。
 - Mesh コライダー: 3Dモデルのポリゴン形状に沿った精密な交差判定。
 - Skeleton コライダー: ボーン階層と連動した部位ごとの球体判定。
- 判定処理の拡張:
 - レイキャスト (Raycast): 高速な視界・射線交差判定。
 - トリガー制御: 押し出しを伴わない `OnTriggerEnter/Stay/Exit`

8. AIシステム：Behavior Tree & Blackboard

- 課題：AIの挙動調整におけるビルド時間のロス
- 対策：ビジュアルノードエディタとデータ駆動
 - **Behavior Tree**: 階層的ノードによる条件分岐と意思決定。
 - **Blackboard**: 個体用メモリを保持し、状態パラメータをノード間で共有。
 - **GUIエディタ**: `imgui-node-editor` でAI構造をリアルタイム編集しJSON出力。



9. 敵AIの行動仕様と動的インタラクション

- **1. Patrol (巡回・監視)**

- ルートパトロール、および首振りによる視界スキャン。

- **2. Investigate (搜索)**

- 銃声や足音（音リソース）の発生源へ移動し、周辺を搜索。

- **3. Chase (戦闘・カバーアクション)**

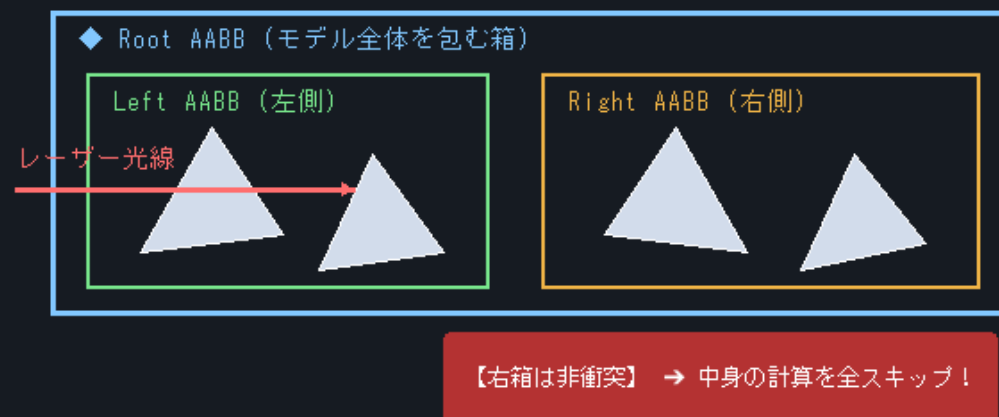
- ターゲット追従。

- **カバー判定:** 射線が通らない「壁の裏側（セーフ領域）」を算出して退避。

- **ピーク射撃:** 遮蔽物から身を乗り出して射撃を行い、クールダウン時は再び隠れる。

10. 経路探索 (NavMesh) と負荷削減 (BVH)

- 障害物の賢い回避と、複数AI動作時の計算最適化
- NavMesh 経路探索
 - 3D地形から歩行可能エリアを抽出し、最短経路 (A^* 探索) をリアルタイム計算。
- BVH (境界ボリューム階層) による枝刈り
 - コライダー群を階層的にAABB (境界箱) でグループ化。
 - 親AABBと非衝突なら、内包される子オブジェクトの判定計算を早期スキップ。



11. デバッグ・計測システム：ボトルネック特定

- **GPUプロファイラ (GpuProfiler)**

- DirectX 12 タイムスタンプクエリをコマンドリストに挿入。
- シャドウパス、メイン描画、ポストエフェクト等のGPU処理時間をミリ秒で正確に測定・ImGuiへ表示。

- **リアルタイムデバッグ描画**

- 3D空間に配置されたコライダーのワイヤーフレームをリアルタイム描画。

- **CPUリークチェッカー & 例外ダンプ生成**

- 起動～終了時のメモリリーク検出、異常終了時のダンプ出力。

12. まとめと今後の展望

- **低レイヤ制御の実践**

- DirectX 12 デバイス、各種バッファ割り当て、およびCPU-GPU間の同期フェンス制御を体系的に設計。

- **コンソール実務レベル of 最適化手法の習得**

- 動的確保の排除、空間分割（ハッシュ・BVH）、連続メモリへのデータ指向アプローチ（DOD）を適用。

- **今後のロードマップ**

- アセットファイルのコンバート（独自バイナリ化）とマルチスレッド非同期ロードの実装。

ご清聴ありがとうございました。